# Lundi 26/10 — IA tutrice (CS50.ai) et fonctionnement du web

Kevin Bardoux

lundi 26 octobre 2026

## Table des matières

Objectifs ..... 2
Matin — Atelier IA tutrice — ouverture du palier 1 · fonctionnement du web : HTTP, DNS, TCP/IP, client/serveur ..... 2
Rappel (5 min) ..... 2
Apport 1 — Le palier 1 : une IA tutrice (10 min) ..... 2
Palier 1 — attention et vérification ..... 4
Créer son compte GitHub (15 min) ..... 4
Prise en main de CS50.ai (30 min) ..... 4
Apport 2 — Le trajet d'une requête (15 min) ..... 5
Trajet — attention et vérification ..... 6
Activité — Le voyage d'une requête (30 min) ..... 6
Si le temps le permet — Un quiz par CS50.ai ..... 7
À retenir — ce matin ..... 7
Débrief ..... 7
Après-midi — TP — « Tuteur socratique » · Learn Prompting, puis « Inspecteur réseau » ..... 7
TP « Tuteur socratique » : bien poser ses questions (1 h) ..... 7
Apport 3 — HTTP : demander et répondre (10 min) ..... 8
HTTP — attention et vérification ..... 10
TP « Inspecteur réseau » (1 h 15) ..... 10
Pas à pas — l'onglet Réseau (10 min) ..... 10
À retenir — cet après-midi ..... 15
Si le temps le permet — Quatre fonctions d'inspecteur ..... 15
Pour aller plus loin ..... 15

Sources ..... 16

> Note
>
> **Compétences visées** : C1 · apprendre en continu · résolution de problème **Livrable** : votre compte GitHub, votre fichier `questions-canard.md` et la fiche « Inspecteur réseau » remplie.

## Objectifs

- Dire ce que le palier 1 autorise, et ce qu'il interdit.
- Poser à CS50.ai une question utile, sans demander la solution.
- Décrire le trajet d'une requête : DNS, TCP/IP, HTTP.
- Lire une requête et sa réponse dans les outils du navigateur.

**Comment se passe la journée.** Le matin, deux notions : l'IA tutrice (palier 1) et le trajet d'une requête. L'après-midi, HTTP, juste avant l'« Inspecteur réseau » qui s'en sert. Chacune suit le même ordre : pourquoi, définition, exemple, pas à pas, erreur fréquente, vérification ; un encadré « À retenir » termine chaque demi-journée.

# Matin — Atelier IA tutrice — ouverture du palier 1 · fonctionnement du web : HTTP, DNS, TCP/IP, client/serveur

## Rappel (5 min)

1. Quand la recherche dichotomique est-elle possible ?
2. Liste deux fois plus longue : que devient $O(n^2)$ ?
3. Combien d'essais pour trouver un nombre de 1 à 100 ?

Trois questions courtes, sans regarder vos notes : la veille, la semaine passée, puis plus loin. Répondez sur une feuille ; la correction suit, à l'oral.

## Apport 1 — Le palier 1 : une IA tutrice (10 min)

- **Pourquoi** : se faire expliquer, sans que l'IA fasse à votre place.
- **CS50.ai** : un tuteur qui guide vers la réponse, sans la donner.

**Pourquoi.** Trois semaines sans IA ont posé vos bases. À partir d'aujourd'hui, une IA peut vous aider à **comprendre** : expliquer une notion, reformuler un message d'erreur. Elle ne doit pas faire le travail à votre place, car le jury vous interrogera sur **votre** code.

**Définition.** Le palier 1 de la charte IA (Formation DWWM 2026-2027 2026) autorise une IA **tutrice** : CS50.ai, le « canard » de Harvard, conçu pour se comporter comme un bon tuteur, qui mène vers la réponse au lieu de la donner (CS50, s. d.). Il s'utilise dans le navigateur, à côté de VS Code. L'autocomplétion par IA de VS Code reste désactivée jusqu'au palier 2b (S16).

**Exemple.** Ce qui est permis, et ce qui ne l'est pas :

```
Oui     expliquer une notion, reformuler un message
d'erreur
Oui     donner un indice, poser des questions, faire un
quiz
Non     donner du code solution ; coller un énoncé
Non     rédiger un texte à rendre
Jamais  un secret, une donnée personnelle, un sujet
d'évaluation
```

**Pas à pas : bien poser une question au canard.** Le formateur pose une question au vidéoprojecteur, en suivant ces étapes ; vous ferez de même pendant la prise en main.

1. **Le contexte** : le langage, l'exercice, ce que vous voulez obtenir.
2. **Ce que vous avez essayé**, et ce qui s'est passé.
3. **Le message d'erreur** exact, s'il y en a un.
4. **Une seule question**, précise.
5. Lisez la réponse : vous a-t-elle guidé, sans faire à votre place ?

Vous avez **10 cœurs**, puis un de plus toutes les 3 minutes : une question précise obtient une réponse utile du premier coup ; « ça marche pas » use trois cœurs. Le canard n'est intégré qu'à l'éditeur en ligne de CS50 (cs50.dev) (CS50, s. d.), pas à votre VS Code. Il reste disponible à tous les paliers suivants. Un usage déclaré et conforme au palier n'est jamais reproché.

CS50.ai est réalisé avec le soutien de Microsoft et d'OpenAI (CS50, s. d.) : vos questions partent vers des services extérieurs. L'équipe de CS50 explique qu'elles sont d'abord débarrassées des données personnelles, puis traitées par GPT-4, hébergé sur Microsoft Azure (Liu et al. 2024) ; la règle ne change pas pour autant.

## Palier 1 — attention et vérification

> ⚠ Attention
>
> Jamais de mot de passe, de clé, de donnée personnelle, ni le code ou l'énoncé d'un sujet d'évaluation dans CS50.ai : vos questions partent vers des services extérieurs.

- **Vérifiez** : « Écris-moi la fonction de l'exercice » : permis ?

## Créer son compte GitHub (15 min)

**Niveau** : guidé.

1. Ouvrez github.com, puis **Sign up**.
2. Utilisez l'adresse e-mail que vous donnerez à Git demain.
3. Choisissez un pseudo professionnel : il apparaîtra sur votre CV.
4. Activez la double authentification.
5. **Réussi si** : vous êtes connecté, double authentification active.

Suivez le guide de création de compte de GitHub (GitHub, s. d.). Le compte sert à vous connecter à CS50.ai, puis vendredi à publier votre premier dépôt. La double authentification protège mieux qu'un mot de passe seul (CNIL, s. d.) : ce sera le sujet de la veille de vendredi.

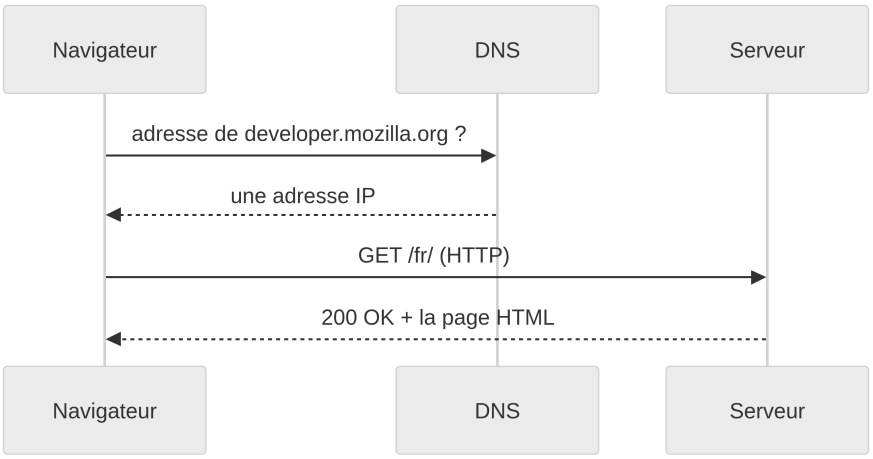
## Prise en main de CS50.ai (30 min)

**Niveau** : guidé.

1. Ouvrez cs50.ai, puis **Log In** avec GitHub.
2. VS Code : créez `questions-canard.md` dans le dossier `dwwm`.
3. Posez une question sur un exercice difficile de la S3.
4. Demandez-lui : « Donne-moi la solution. » Notez sa réponse.
5. **Réussi si** : deux questions et leurs réponses dans le fichier.

Décrivez l'exercice avec vos mots : ne collez pas l'énoncé entier. Le canard est conçu pour guider sans donner la solution (CS50, s. d.) : à l'étape 4, il doit vous renvoyer vers une piste ou une question, pas vers du code. Il répond en français si vous lui écrivez en français.

## Apport 2 — Le trajet d'une requête (15 min)




D'abord le DNS traduit le nom en adresse IP ; puis la requête HTTP part vers cette adresse, et le serveur répond.

**Pourquoi.** Vous allez construire des sites et des API : savoir ce qui se passe entre « je tape une adresse » et « la page s'affiche » permet de comprendre une erreur, un site lent ou une faille.

**Définition.** Le **client** (votre navigateur) demande une ressource ; le **serveur** (un ordinateur distant) la renvoie. Entre les deux, trois étapes : le **DNS** (*Domain Name System*) traduit le nom de domaine en adresse IP (MDN Web Docs, s. d.-d) ; une **adresse IP** identifie un appareil sur un réseau (MDN Web Docs, s. d.-a) ; **TCP/IP** transporte les données : IP achemine des paquets d'une machine à l'autre, TCP établit une connexion et vérifie que tout arrive, dans l'ordre (MDN Web Docs, s. d.-f). HTTP s'appuie sur cette connexion. Pour une vue d'ensemble, voir « Comment fonctionne le Web » sur MDN (MDN Web Docs, s. d.-c).

**Une image, avec sa limite.** Le DNS fonctionne comme un annuaire : on cherche un nom, on obtient un numéro. Limite de

l'image : plusieurs serveurs DNS se répartissent le travail, et la réponse est gardée en mémoire un moment.

**Pas à pas.** Suivez le schéma ligne par ligne :

1. Vous tapez `developer.mozilla.org` dans le navigateur.
2. Le navigateur demande au DNS l'adresse IP de ce nom.
3. Le DNS répond par une adresse IP.
4. Le navigateur ouvre une connexion TCP vers cette adresse.
5. Il envoie la requête HTTP `GET /fr/`.
6. Le serveur répond `200 OK`, avec la page HTML ; le navigateur l'affiche.

Pour aller plus loin, facultatif : la fiche de lecture Le Web (fonctionnement, histoire et enjeux, environ 15 minutes).

## Trajet — attention et vérification

> ⚠ Attention
>
> Le nom de domaine (`developer.mozilla.org`) n'est pas l'adresse IP : c'est le DNS qui fait le lien. Sans réponse du DNS, aucune page ne s'affiche.

- **Vérifiez** : qui traduit un nom de domaine en adresse IP ?

## Activité — Le voyage d'une requête (30 min)

**Niveau** : semi-guidé.

1. En groupe : « je tape une adresse, la page s'affiche ».
2. Un **dessinateur** au tableau, un **narrateur**, un **vérificateur**.
3. Dessinez chaque étape : DNS, TCP, HTTP, affichage.
4. Demandez à CS50.ai : « Pose-moi 3 questions sur ce trajet. »
5. **Réussi si** : ses questions sont justes, et il n'a pas donné les réponses.

Le **dessinateur** trace les étapes au tableau ; le **narrateur** les raconte à voix haute ; le **vérificateur** compare avec le schéma de l'apport 2 et signale un oubli. Si vous préférez ne pas parler devant le groupe, prenez le rôle de dessinateur ou de vérificateur.

**Si le temps le permet — Un quiz par CS50.ai**

1. Demandez à CS50.ai un quiz de 5 questions sur le trajet d'une requête.
2. Répondez sans regarder le support.
3. Vérifiez vos réponses sur MDN.

### À retenir — ce matin

> À retenir
>
> - **Palier 1** : CS50.ai explique et guide ; jamais de solution, jamais de secret.
> - **Trajet** : DNS (nom → adresse IP), TCP/IP (transport), HTTP.

Le détail, notion par notion :

- **Palier 1** : l'IA tutrice aide à comprendre ; une bonne question donne le contexte, ce que vous avez essayé, le message exact, une seule question ; jamais de secret ni de sujet d'évaluation.
- **Trajet d'une requête** : le DNS traduit le nom en adresse IP ; TCP/IP transporte les données ; HTTP porte la requête et la réponse.

### Débrief

- Qu'a répondu le canard quand vous avez demandé la solution ?
- Qui traduit un nom de domaine en adresse IP ?
- Dans quel ordre interviennent DNS, TCP/IP et HTTP ?

## Après-midi — TP — « Tuteur socratique » · Learn Prompting, puis « Inspecteur réseau »

### TP « Tuteur socratique » : bien poser ses questions (1 h)

**Niveau** : semi-guidé.

1. Trois situations (ci-dessous), avec CS50.ai.
2. Pour chacune : une question v1, puis une v2 améliorée.

3. Comparez : la réponse est-elle meilleure ? Le canard a-t-il guidé ?
4. Notez tout dans `questions-canard.md`.
5. Lisez l'introduction de Learn Prompting : notez une idée.

Les trois situations :

1. **Comprendre une erreur** : collez le message `TypeError: Cannot read properties of undefined (reading 'length')` (vu en semaine 3) et demandez au canard de vous aider à comprendre, sans corriger à votre place.
2. **Comprendre une notion** : « Je ne comprends pas la portée des variables. »
3. **Être débloqué** : décrivez avec vos mots (sans coller l'énoncé) un exercice de la semaine 3 qui vous a posé problème, et demandez un indice.

Pour chaque situation, écrivez d'abord la question telle qu'elle vous vient, puis une version 2 qui suit « Bien poser une question au canard » (contexte, ce que vous avez essayé, message exact, une seule question). Comparez les deux réponses. Learn Prompting est un guide en ligne pour apprendre à formuler des instructions à une IA (Learn Prompting, s. d.) : retenez une idée qui améliore vos questions.

## Apport 3 — HTTP : demander et répondre (10 min)

- **Pourquoi** : toute page, image ou donnée passe par HTTP.
- **HTTP** : une requête (méthode, chemin), une réponse (code, contenu).

**Pourquoi.** Chaque page, chaque image, chaque donnée d'une application web arrive par une requête HTTP. Lire une requête et sa réponse est le premier geste pour comprendre un problème.

**Définition.** HTTP est le protocole des échanges entre client et serveur : le client envoie une **requête** (une méthode comme `GET`, un chemin, des en-têtes), le serveur renvoie une **réponse** (un code, des en-têtes, puis le contenu) (MDN Web Docs, s. d.-g). HTTPS est la version chiffrée de HTTP (MDN Web Docs, s. d.-e).




Figure 1. – Une requête part du navigateur vers le serveur (méthode, adresse) ; la réponse revient (code, puis le corps : ici, la page HTML).

**Exemple.** La même requête, puis sa réponse, telles qu'elles s'écrivent :

```
GET /fr/ HTTP/1.1
Host: developer.mozilla.org

HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
```

**Pas à pas.** Lisez l'exemple ligne par ligne :

1. `GET` : la méthode, « je veux lire ».
2. `/fr/` : le chemin de la ressource demandée.
3. `Host` : le nom du site.
4. `200 OK` : le code de la réponse, ici un succès.
5. `Content-Type` : la sorte de contenu renvoyé, ici une page HTML.

Les codes de statut sont regroupés en cinq classes : réponses informatives (100–199), succès (200–299), redirections (300–399, par exemple `301`, `304`), erreurs du client (400–499, `404` : introuvable) et erreurs du serveur (500–599, `500`) (MDN Web Docs, s. d.-b).

## HTTP — attention et vérification

> ⚠ Attention
>
> `4xx` : la requête est en cause (adresse fausse, accès refusé).
> `5xx` : le serveur est en cause. Lisez la famille du code avant de chercher.

- **Vérifiez** : que signifie `404` ? Et `500` ?

## TP « Inspecteur réseau » (1 h 15)

**Niveau** : semi-guidé.

1. Ouvrez MDN, puis les outils du navigateur (F12), onglet **Réseau**.
2. Rechargez sans cache (`Cmd+Shift+R` (mac), `Ctrl+F5` (windows)).
3. Cliquez sur la première : méthode, code, en-têtes.
4. Remplissez la fiche d'inspection (ci-dessous).
5. Ouvrez une page qui n'existe pas : notez le code.

L'onglet Réseau affiche toutes les requêtes d'une page, avec leur méthode, leur code et leurs en-têtes : voir la documentation de Firefox (Mozilla, s. d.) ou de Chrome (Google, s. d.).

**Fiche d'inspection**, à recopier et remplir :

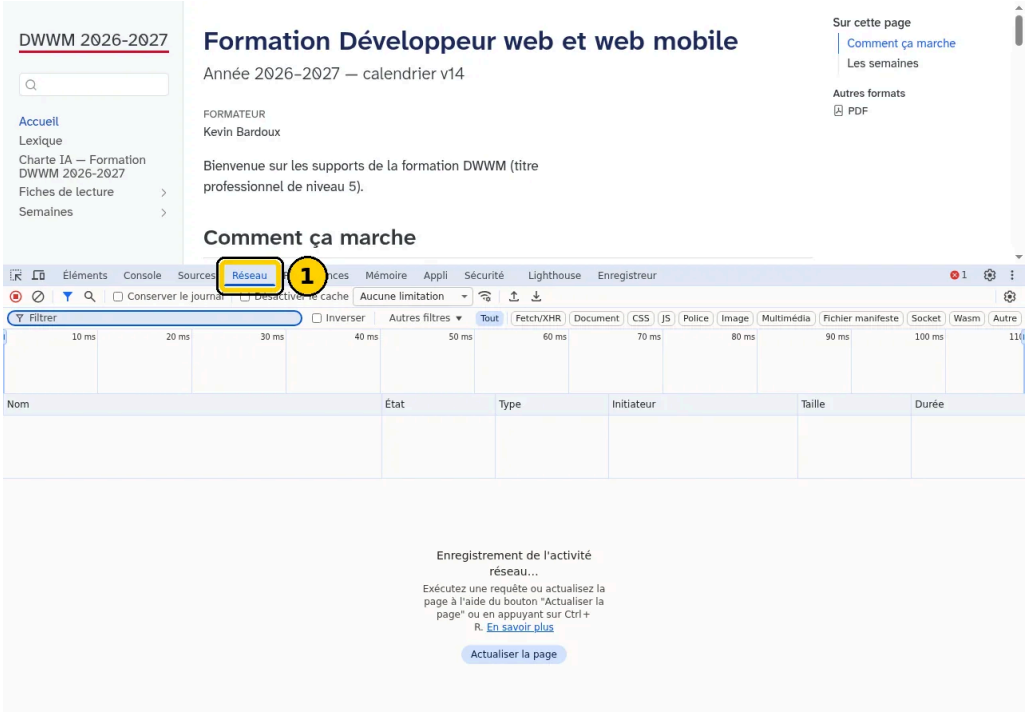
| Question | Votre réponse |
|---|---|
| Combien de requêtes pour charger la page ? | |
| Méthode et code de la première requête (le document) | |
| Valeur de l'en-tête `Content-Type` de cette réponse | |
| Une requête d'image : son code et sa taille | |
| Le site utilise-t-il HTTPS ? Comment le voyez-vous ? | |
| Code obtenu pour une page qui n'existe pas | |

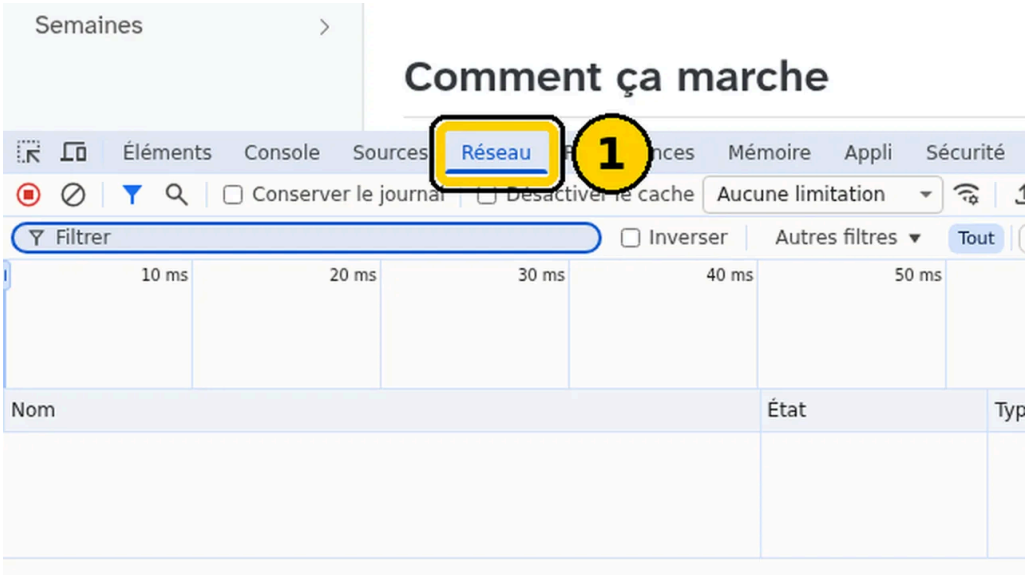
### Pas à pas — l'onglet Réseau (10 min)

Chaque étape donne une action, puis ce que vous devez voir. Sur chaque capture, l'endroit à regarder ou à cliquer est entouré d'un cadre jaune, avec le numéro de l'étape ; un zoom sur la zone utile suit chaque capture. Les captures montrent le site du cours ; sur

MDN, l'écran est le même, avec d'autres lignes. Sur téléphone, touchez une capture pour l'agrandir.

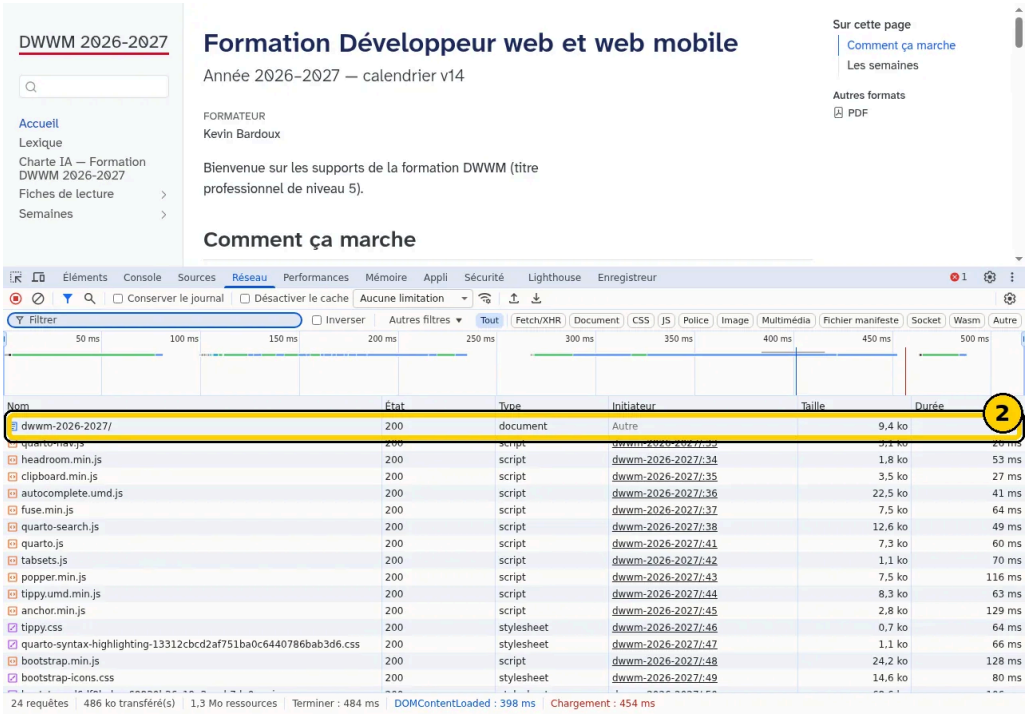
**Étape 1.** Ouvrez la page, appuyez sur `F12`, puis cliquez sur l'onglet **Réseau**.

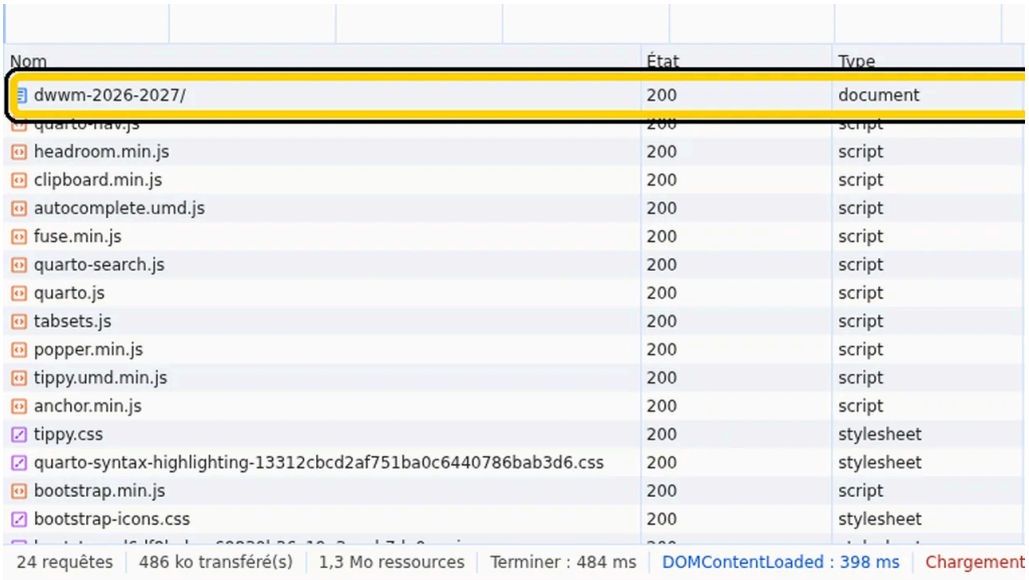



**Vous devez voir** : une liste vide et le message « Enregistrement de l'activité réseau… ».

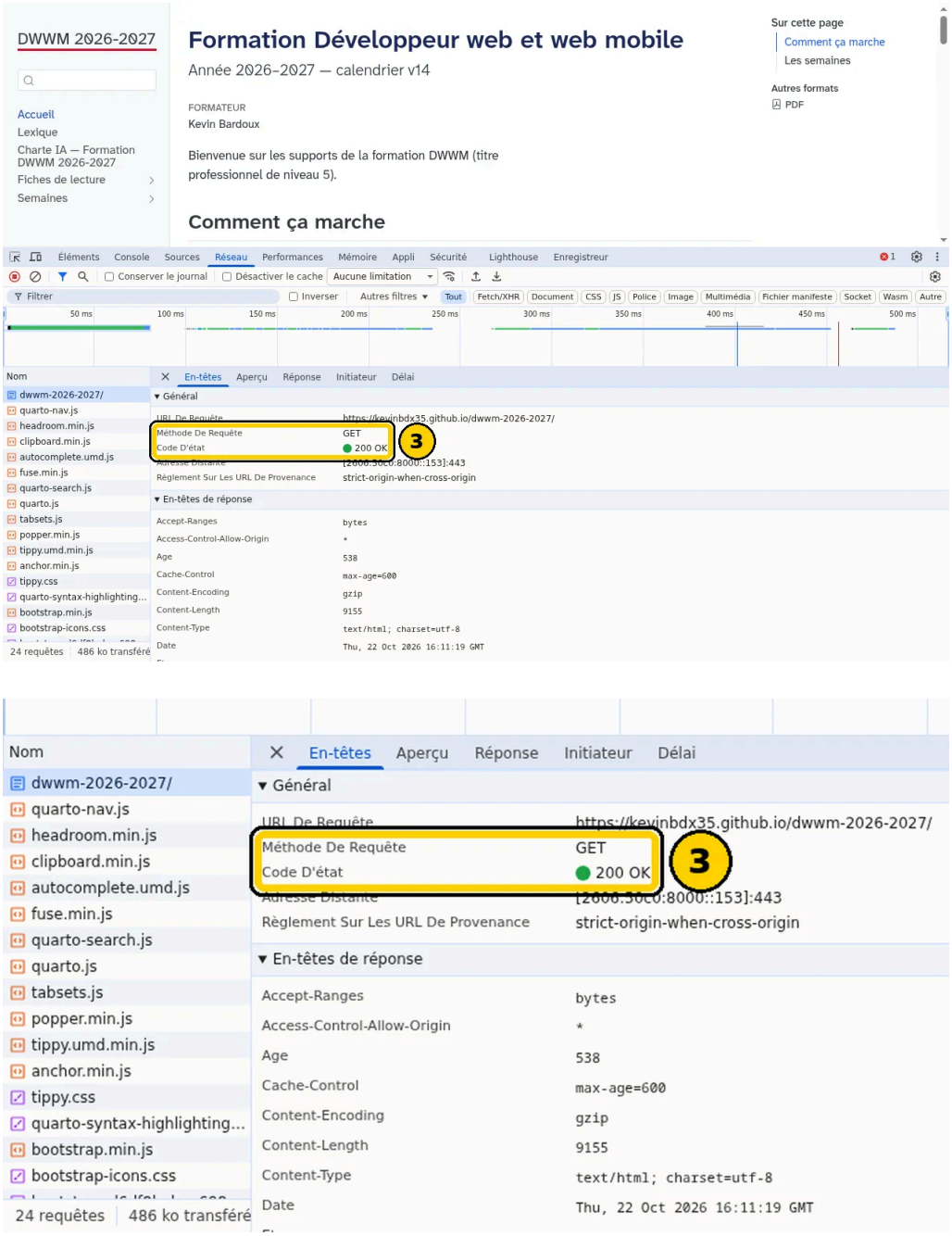
**Étape 2.** Rechargez la page sans cache : `Cmd+Shift+R` (mac), `Ctrl+F5` (windows).




**Vous devez voir** : une ligne par requête. La première est le document (la page HTML), avec `200` dans la colonne **État**.

**Étape 3.** Cliquez sur le nom de la première ligne.

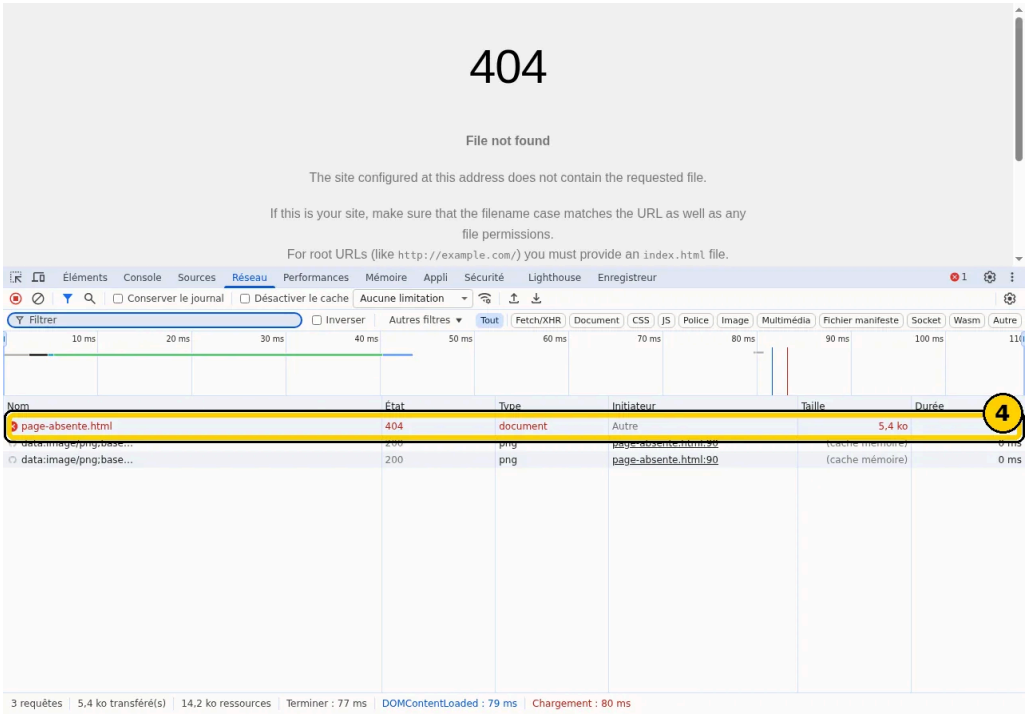


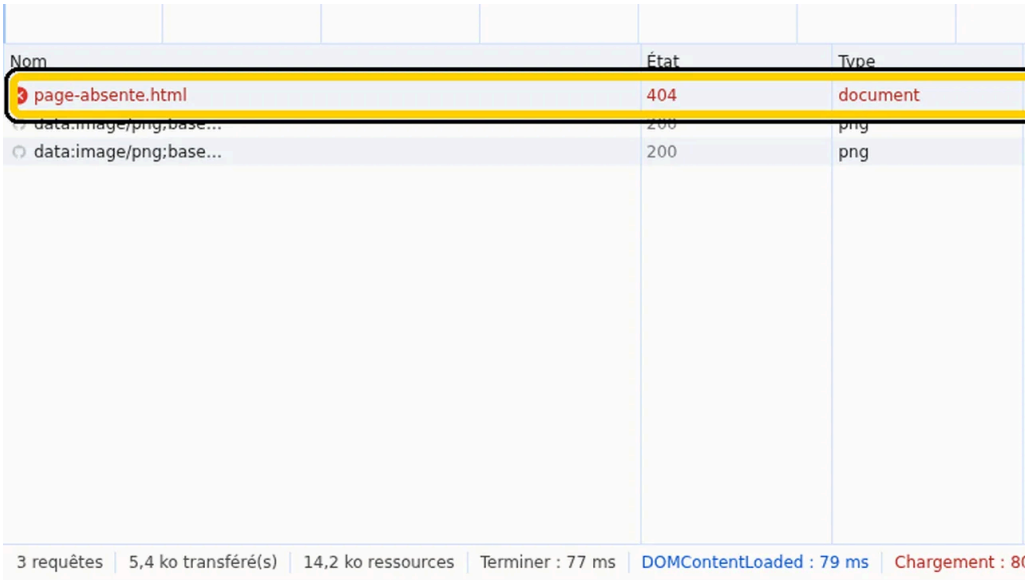




**Vous devez voir** : la méthode `GET` et le code `200 OK` ; plus bas, les en-têtes de réponse, dont `Content-Type`.

**Étape 4.** Dans la barre d'adresse, ajoutez `page-absente.html` à la fin de l'adresse, puis appuyez sur `Entrée`.




**Vous devez voir** : la ligne du document en rouge, avec `404` dans la colonne **État**.

Si vous rechargez avec `F5` seul, la première ligne peut afficher `304` : le serveur répond « pas modifié », et le navigateur réutilise la copie qu'il a gardée.

## À retenir — cet après-midi

> À retenir
>
> - **Question v2** : contexte, essai, message exact, une seule question.
> - **Onglet Réseau** (F12) : chaque requête, sa méthode, son code.
> - **HTTP** : requête (méthode, chemin), réponse (code) ; HTTPS chiffre.

Le détail : `GET` lit une ressource ; `2xx` succès, `3xx` redirection, `4xx` erreur du client, `5xx` erreur du serveur ; HTTPS chiffre l'échange, mais ne garantit pas l'honnêteté du site.

### Critères de réussite

- □ Votre compte GitHub est créé, avec la double authentification
- □ `questions-canard.md` : trois questions, en version 1 et 2
- □ Le canard a guidé au lieu de donner la solution
- □ La fiche « Inspecteur réseau » est remplie

### Si le temps le permet — Quatre fonctions d'inspecteur

- **i1-code-statut** : la famille d'un code HTTP.
- **i2-url-securisee** : l'adresse utilise-t-elle HTTPS ?
- **i3-hote** : le nom d'hôte d'une adresse.
- **i4-parametre** : la valeur d'un paramètre de l'adresse.
- Archive de la semaine, tests fournis : `node --test i1-code-statut.test.js` (et ainsi de suite).

Les fichiers sont dans l'archive de la semaine. L'objet `URL` découpe une adresse en morceaux (MDN Web Docs, s. d.-h) : `protocol` (MDN Web Docs, s. d.-k), `hostname` (MDN Web Docs, s. d.-j), et `searchParams.get` pour les paramètres (MDN Web Docs, s. d.-l) ; `URL.canParse` dit si un texte est une adresse valide (MDN Web Docs, s. d.-i). Ces trois dernières pages sont en anglais.

### Pour aller plus loin

- Dans le terminal : `nslookup developer.mozilla.org`.
- Quelle adresse IP obtenez-vous ? Et votre voisin ?

## Sources


CNIL. s. d. « Mots de passe : recommandations pour maîtriser sa sécurité ». Consulté le 26 septembre 2026. https://www.cnil.fr/fr/mots-de-passe-des-recommandations-de-securite-minimales-pour-les-entreprises-et-les-particuliers.

CS50. s. d. « CS50.ai — CS50 Docs ». Consulté le 28 septembre 2026. https://cs50.readthedocs.io/cs50.ai/.

Formation DWWM 2026-2027. 2026. *Charte IA de la formation*.

GitHub. s. d. « Creating an account on GitHub ». Consulté le 26 septembre 2026. https://docs.github.com/en/get-started/start-your-journey/creating-an-account-on-github.

Google. s. d. « Inspect network activity — Chrome DevTools ». Consulté le 26 septembre 2026. https://developer.chrome.com/docs/devtools/network.

Learn Prompting. s. d. « Bienvenue (introduction) ». Consulté le 26 septembre 2026. https://learnprompting.org/fr/docs/introduction.

Liu, Rongxin, Carter Zenke, Charlie Liu, Andrew Holmes, Patrick Thornton, et David J. Malan. 2024. « Teaching CS50 with AI: Leveraging Generative Artificial Intelligence in Computer Science Education ». *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1 (SIGCSE 2024)*, 750-56. https://doi.org/10.1145/3626252.3630938.

MDN Web Docs. s. d.-a. « Adresse IP - Glossaire ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Glossary/IP_Address.

MDN Web Docs. s. d.-b. « Codes de statut de réponse HTTP - HTTP ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/HTTP/Reference/Status.

MDN Web Docs. s. d.-c. « Comment fonctionne le Web - Apprendre ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Learn_web_development/Getting_started/Web_standards/How_the_web_works.

MDN Web Docs. s. d.-d. « DNS - Glossaire ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Glossary/DNS.

MDN Web Docs. s. d.-e. « HTTPS - Glossaire ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Glossary/HTTPS.

MDN Web Docs. s. d.-f. « TCP - Glossaire ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Glossary/TCP.

MDN Web Docs. s. d.-g. « Un aperçu de HTTP - HTTP ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/HTTP/Guides/Overview.

MDN Web Docs. s. d.-h. « URL - Les API Web ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/API/URL.

MDN Web Docs. s. d.-i. « URL: canParse() static method - Web APIs ». Consulté le 26 septembre 2026. https://developer.mozilla.org/en-US/docs/Web/API/URL/canParse_static.

MDN Web Docs. s. d.-j. « URL: hostname property - Web APIs ». Consulté le 26 septembre 2026. https://developer.mozilla.org/en-US/docs/Web/API/URL/hostname.

MDN Web Docs. s. d.-k. « URL.protocol - Les API Web ». Consulté le 26 septembre 2026. https://developer.mozilla.org/fr/docs/Web/API/URL/protocol.

MDN Web Docs. s. d.-l. « URLSearchParams: get() method - Web APIs ». Consulté le 26 septembre 2026. https://developer.mozilla.org/en-US/docs/Web/API/URLSearchParams/get.

Mozilla. s. d. « Network Monitor — Firefox Source Docs ». Consulté le 26 septembre 2026. https://firefox-source-docs.mozilla.org/devtools-user/network_monitor/.